DSA Question

1. Two Sum

Problem Statement: Find the two numbers that add up to a given target sum in an array. **Solution:**

```
import java.util.HashMap;

class Solution {
    public int[] twoSum(int[] nums, int target) {
        HashMap<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (map.containsKey(complement)) {
                return new int[] { map.get(complement), i };
            }
            map.put(nums[i], i);
        }
        return new int[] {};
    }
}
```

2. Valid Parentheses

Problem Statement: Check if a given expression has balanced parentheses. **Solution:**

```
import java.util.Stack;

class Solution {
    public boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();
        for (char c : s.toCharArray()) {
            if (c == '(' || c == '{' || c == '[') {
                stack.push(c);
            } else {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if ((c == ')' && top != '(') ||
                    (c == '}' && top != '{') ||
                    (c == ']' && top != '[')) {
                    return false;
                }
            }
        }
        return stack.isEmpty();
    }
}
```

3. Reverse a Linked List

Problem Statement: Reverse a singly linked list. **Solution:**

```
class ListNode {
   int val;
   ListNode next;
   ListNode(int x) { val = x; }
}

class Solution {
   public ListNode reverseList(ListNode head) {
      ListNode prev = null;
      ListNode curr = head;
      while (curr != null) {
         ListNode nextTemp = curr.next;
         curr.next = prev;
         prev = curr;
         curr = nextTemp;
      }
      return prev;
   }
}
```

4. Merge Two Sorted Lists

Problem Statement: Merge two sorted linked lists into a single sorted list. **Solution:**

```
class Solution {
   public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
      ListNode dummy = new ListNode(-1);
      ListNode current = dummy;

      while (l1 != null && l2 != null) {
         if (l1.val <= l2.val) {
            current.next = l1;
            l1 = l1.next;
         } else {
            current.next = l2;
            l2 = l2.next;
         }
         current = current.next;
      }
      current.next = (l1 != null) ? l1 : l2;
      return dummy.next;
   }
}
```

5. Missing Number

Problem Statement: Find the missing number in a sorted array. **Solution:**

```
class Solution {
    public int missingNumber(int[] nums) {
        int n = nums.length;
        int expectedSum = n * (n + 1) / 2;
        int actualSum = 0;
        for (int num : nums) {
            actualSum += num;
        }
        return expectedSum - actualSum;
    }
}
```

6. Reverse String

Problem Statement: Reverse a given string in-place. **Solution:**

```
class Solution {
    public void reverseString(char[] s) {
        int left = 0;
        int right = s.length - 1;
        while (left < right) {
            char temp = s[left];
            s[left] = s[right];
            s[right] = temp;
            left++;
            right--;
        }
    }
}
```

7. Lowest Common Ancestor of a Binary Search Tree

Problem Statement: Find the lowest common ancestor of two nodes in a BST.

Solution:

```
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int x) { val = x; }
}

class Solution {
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode
q) {
        while (root != null) {
            if (p.val < root.val && q.val < root.val) {
                root = root.left;
            } else if (p.val > root.val && q.val > root.val) {
```

```
            root = root.right;
        } else {
            return root;
        }
    }
    return null;
  }
}
```

8. Trapping Rain Water

Problem Statement: Given n non-negative integers representing an elevation map, compute how much water it can trap after raining. **Solution:**

```
class Solution {
  public int trap(int[] height) {
    if (height == null || height.length == 0) return 0;
    int left = 0, right = height.length - 1;
    int leftMax = 0, rightMax = 0;
    int water = 0;

    while (left < right) {
      if (height[left] < height[right]) {
        if (height[left] >= leftMax) leftMax = height[left];
        else water += leftMax - height[left];
        left++;
      } else {
        if (height[right] >= rightMax) rightMax = height[right];
        else water += rightMax - height[right];
        right--;
      }
    }
    return water;
  }
}
```

9. Longest Consecutive Sequence

Problem Statement: Given an unsorted array of integers, find the length of the longest consecutive elements sequence. **Solution:**

```
import java.util.HashSet;
import java.util.Set;

class Solution {
  public int longestConsecutive(int[] nums) {
    Set<Integer> numSet = new HashSet<>();
    for (int num : nums) {
      numSet.add(num);
    }
```

```
        int longestStreak = 0;
        for (int num : numSet) {
            if (!numSet.contains(num - 1)) {
                int currentNum = num;
                int currentStreak = 1;

                while (numSet.contains(currentNum + 1)) {
                    currentNum += 1;
                    currentStreak += 1;
                }
                longestStreak = Math.max(longestStreak, currentStreak);
            }
        }
        return longestStreak;
    }
}
```

10. Minimum Window Substring

Problem Statement: Given strings S and T, find the minimum window in S which will contain all the characters in T. **Solution:**

```
import java.util.HashMap;
import java.util.Map;

class Solution {
    public String minWindow(String s, String t) {
        if (s.length() == 0 || t.length() == 0) return "";

        Map<Character, Integer> dictT = new HashMap<>();
        for (int i = 0; i < t.length(); i++) {
            dictT.put(t.charAt(i), dictT.getOrDefault(t.charAt(i), 0) + 1);
        }

        int required = dictT.size();
        int l = 0, r = 0, formed = 0;
        Map<Character, Integer> windowCounts = new HashMap<>();
        int[] ans = {-1, 0, 0};

        while (r < s.length()) {
            char c = s.charAt(r);
            windowCounts.put(c, windowCounts.getOrDefault(c, 0) + 1);

            if (dictT.containsKey(c) && windowCounts.get(c).intValue() ==
dictT.get(c).intValue()) {
                formed++;
```

```
            }

            while (l <= r && formed == required) {
                c = s.charAt(l);
                if (ans[0] == -1 || r - l + 1 < ans[0]) {
                    ans[0] = r - l + 1;
                    ans[1] = l;
                    ans[2] = r;
                }

                windowCounts.put(c, windowCounts.get(c) - 1);
                if (dictT.containsKey(c) && windowCounts.get(c).intValue() <
dictT.get(c).intValue()) {
                    formed--;
                }
                l++;
            }
            r++;
        }
        return ans[0] == -1 ? "" : s.substring(ans[1], ans[2] + 1);
    }
}
```